

A look at the ChaCha stream cipher

containedreality

Description

Today I'll be taking a quick look at the [ChaCha](#) stream cipher.

Copyright

Copyright (C) 2026 by [containedreality](#). Licensed under [CC BY-SA 4.0](#).

Table of Contents

Introduction to ChaCha.....	2
Usage and Adoption.....	2
Cipher Design.....	2
State.....	2
Rounds.....	3
Closing Notes.....	4
References.....	4
Implementations.....	4

Introduction to ChaCha

ChaCha was invented by Daniel J. Bernstein. We will be looking at the version of ChaCha documented in [RFC-8439](#). It is commonly used with Poly1305 for AEAD, we will not be looking at Poly1305 here. ChaCha20 is based off of [Salsa20](#).

Usage and Adoption

ChaCha20 used in [WireGuard](#) and [TLSv1.3](#) (where it's also alongside AES-128-GCM and AES-256-GCM). ChaCha20 is in favor when a stream cipher is acceptable and there may not be AES hardware acceleration or when the cipher has to be written in software.

Cipher Design

ChaCha or in this case ChaCha20 is a stream cipher consisting of 20 rounds, a 96 bit nonce, 256 bit key, and 32 bit counter.

ChaCha rounds can vary so you may see ChaCha20, ChaCha12 or even ChaCha8.

ChaCha has a ARX (add-rotate-xor)-like design which is used to produce psuedo-random numbers and it works on 32-bit little endian integers. In C the quarter round would be:

```
#define ROTL32(x, n) ((x << n) | (x >> (32 - n)))

#define CHACHA_QR(a, b, c, d) { a += b; d ^= a; d = ROTL32(d,16);\
    c += d; b ^= c; b = ROTL32(b, 12);\
    a += b; d ^= a; d = ROTL32(d, 8);\
    c += d; b ^= c; b = ROTL32(b, 7); }
```

This makes the cipher lightweight and easier to implement in software securely as there are no look-up tables that can cause things like timing and cache side-channel attacks.

State

As before we learned that ChaCha uses a state and mixes it together with the important information, we went into it a bit in the cipher design. This is supposed to be a more indepth explanation of the state. The ChaCha state is 512 bits in size meaning with 2^{32}

counters per key and therefore $4294967296 * 512$, we have ~256 GiB to encrypt with a single key, but to be conservative I'd rekey much before that.

So the ChaCha state consists of sixteen 32 bit unsigned little endian integers, and the layout looks kinda like the following:

CONSTANT	CONSTANT	CONSTANT	CONSTANT
KEY	KEY	KEY	KEY
KEY	KEY	KEY	KEY
COUNTER	NONCE	NONCE	NONCE

And it gets all mixed mixed together many times by the rounds.

The constant is “expand 32-byte k” in ASCII. The counter is just something that counts up to keep encrypting data with the same key+nonce pair. The nonce ensures that the keystream will be unique and enables us to send multiple messages with the same key without worrying about keystream reuse.

Rounds

The rounds mix up the state nicely into a nice psuedo-random keystream that can be used for encrypting messages.

ChaCha20 has 20 rounds in total, and it runs the column and diagonal rounds times. So $2 * 10 = 20$ rounds. First to be ran 1-4 are the column quarter rounds and then it has 5-8 even quarter diagonal rounds. For example:

```
for(int i = 0; i < 10; i++) {
    // column round
    CHACHA_QR(state[0], state[4], state[8] , state[12]);
    CHACHA_QR(state[1], state[5], state[9] , state[13]);
    CHACHA_QR(state[2], state[6], state[10], state[14]);
    CHACHA_QR(state[3], state[7], state[11], state[15]);

    // diagonal round
    CHACHA_QR(state[0], state[5], state[10], state[15]);
    CHACHA_QR(state[1], state[6], state[11], state[12]);
    CHACHA_QR(state[2], state[7], state[8] , state[13]);
    CHACHA_QR(state[3], state[4], state[9] , state[14]);
}
```

After this the state should be nicely mixed together.

Then after everything is done we should get out final ChaCha state or keystream that can be used to encrypt messages by: **data xor state**. Remember to increment the COUNTER after each 64 bytes encrypted.

Closing Notes

This for the most part, only covered ChaCha specifically ChaCha20 but there are other ChaCha variants like XChaCha20 which features a larger nonce. You should also look into pairing it with something like Poly1305 if you need AEAD functionality.

References

- [RFC-8439](#)
- [My personal ChaCha implementation](#)

Implementations

- [My personal ChaCha implementation \(use with caution\)](#)
- [OpenSSL](#)
- [Libsodium](#)